ECE459: Programming for Performance Winter 2027

Lecture 29 — Frontend Performance

Jeff Zarnett *2026-11-11*

Feelings and Facts

Most topics of the course focus on how to actually improve the execution speed of the work to be done, which could be measured in either CPU time or wall clock time reductions (or, even better, both!). Frontend performance is unlike those kinds of improvements, because the user's feelings matter. If they *perceive* the page as fast, that's as good as, or better than, the page actually being faster.

Broadly, three main things help the user perceive the page as fast. Acknowledgement of their actions is a big one: showing the user that their click has done something right away leads to a lot less frustration. You might have heard me [JZ] tell the story about how slow acknowledgement of the purchase action caused me to buy two train tickets when I only needed one. The second thing users want is an indication of progress: show a loading bar, a spinner, or some other sign that makes it clear that the thing they asked for is being worked on (or at least in a queue). The last thing is the most controversial: make the user think it's done when it's not. Work that is completed after the user thinks the page is finished loading is "free".

Let's talk about the thresholds that matter to users:

1. Under 0.1 seconds is "instant".
2. Between 1 and 10 seconds, the delay is noticeable, so we need some sort of loading spinner.
3. After 10 seconds, the user is impatient, so we need to try to find a way to end the waiting before the work is done. If we can't do that, then don't keep the user waiting: tell them we'll notify them when it's ready, or send them the result when it's done.

Motivation. The user experience matters here. Aside from the correctness issues like buying two train tickets, there's also the risk that customers just... stop using your app or abandon whatever they were doing. The absence of complaints about responsiveness is noted in the surveys, including a source mentioned earlier [KSNH15]. We could interpret this positively, as in, performance is not a major concern. Or, users who give up don't bother to write reviews.

If we look at a study on video delays [KS12], users start to abandon a video load if it does not start within two seconds. They also cite sources saying users abandon a website if the page does not load in 4 seconds and that even a delay of 400 ms is correlated with lower usage. Faster internet speeds make this problem worse, because users accustomed to speed will not wait for slow pages, though their willingness to wait is moderated by their device: mobile network waiting is tolerated better than fibre connections. What the user is waiting for may also play a role: if the data needed is small, the user's patience is small. If I just want to get the train departure times for the Kitchener GO line from Union Station in Toronto, I might be in a big hurry, but even if I'm not, this is a small amount of data and should take practically no time to deliver. If I asked for an analysis of all engineering students' performance on midterms in the last five years, yes, I can expect to wait a bit longer and won't be upset.

App store reviews being insufficient to motivate this, let the money do the talking: Deloitte Digital and 55 showed that a 0.1 second improvement in 15 retail brands' mobile website resulted in 8.4% more conversions and 9.2% higher order value [D20]. Actually, the whole source document goes over the impact of speed improvements on purchase behaviour across different kinds of products: retail, travel, luxury purchases, but also things like lead generation.

Trickery of Timing

If the goal is to make the user feel as though the work is done sooner, what has to happen? We can't just lie and say "it's totally done, you just can't see it yet!" like an eight-year-old who forgot to do his math homework. We'll also exclude from discussion things that look like only loading the first n rows and then loading the rest when the user scrolls down. Incremental or paginated loading is a good idea, of course, but it does fall firmly under "do less work", so that can be done independently of the ideas in this topic.

Before we think about anything more exotic, let's look at showing a partial page as soon as we can. There are levels to this – we can show a page that just says things are loading, or show partial results (or both). Let's start with loading the shell. It's the frame around where the content will go, while we wait for the content to be delivered. If I'm a student who wants to see my grades in Quest, I am there for the numbers, but if the frame, navigation, and everything around where the grades go has loaded quickly, I at least feel like something is happening.

A couple quick definitions from Google's `web.dev` pages [Wal23, WP25]. The *first contentful paint (FCP)* is the time it takes before any part of the page's content is shown on the screen (this can include text, images, background images, etc). The *largest contentful paint (LCP)* is the time when the biggest element that will be visible (usually the main content) is drawn on the screen. Unless a page is super simple, these are not going to be the same: there will be some user-visible time between the first and largest contentful paints.

FCP and LCP are measured as percentiles and people may choose the 75th or 90th percentile times for page loads in goal-setting. This helps even out some of the variability that naturally exists in computing. It also accounts for things like some customers, cohorts, or train schedules that are larger or smaller than others. Faster is better for both of these, though we can see from earlier that 4 seconds seems to be a limit of user patience and 4.0 seconds is the threshold of "poor" performance in the eyes of Google and 2.5 seconds is good. FCP good target is 1.8 seconds.

The FCP is a nice thing that keeps the user happy as the data they came here for is being loaded. But just showing that shell without the content is going to lead to unhappy users: they're here to see the content, not just the frame (or the ads). So we need to make the LCP happen faster. Obviously, we want to start the loading of that data as early as possible, and we want to render it quickly when the content arrives.

The `web.dev` analysis breaks the LCP down into four subparts that execute sequentially with no gaps; optimizing any of these hopefully reduces the total loading time, though there is a risk it just shifts the time to another part. There's the time to first byte (time from the start of the page load to the first byte of the HTML response), resource load delay (time between the first byte and the start of loading of the LCP resource), resource load duration (how long it takes to get the LCP data), and element render delay (how long it takes to draw the LCP content).

Improving the time to first byte is difficult because it is partly dependent on the network speed, though it has a lot of backend elements like server response time and redirects. But the resource load delay is something we can address: start the loading earlier for the LCP data. The loading time for that data may be dependent on the backend (see other course topics about that). Finally, improving rendering of the data, when it arrives, also helps. The important insight of the LCP thinking is that the user considers things "done" when the LCP element is present, even if there are more things to load.

While it might be obvious to the user what they came here for, how does their browser know what the LCP content is? In a way, it doesn't, unless we give a hint (which is possible in the HTML code). The hint is used by a browser feature called the preload scanner. The preload scanner exists because the main parser blocks while it loads, parses, or renders certain elements in a sequential manner. The goal of preload scanning is to look ahead in the HTML the browser receives to identify resources that are needed, so it can request them sooner and they're available for rendering [Wag22]. This work is alongside the regular loading and rendering pipeline, so it is doing some additional work, but it's worth it if it kicks off the request earlier because then we get the LCP content sooner. The hint is, of course, not always necessary, since the preload scanner can find the content – if it is in the HTML and not loaded in later by some Javascript – but the hint can help it prioritize the LCP element [WP25].

Life is a Waterfall

One of the common pitfalls that leads us down the path of slow LCP renders is something we call a *waterfall*, which is according to the ReactJS documentation "an unintentional sequence that should have been parallelized". This happens when the code fetches some data, renders it, which prompts another fetch and another render, then another fetch, and another render, and so on. If we fetched all the data up-front it would go a lot faster!

Cooking analogy: imagine you are cooking a recipe that consists of multiple ingredients. If you just look at the instructions one at a time, as you read each step, you will sometimes discover that you need a new ingredient or resource. If step one is chop the onion, you go get the onion. If step two is put oil in the pan and heat it up, you go get the oil. Then you add the onion but you have to get salt... You can see how this is inefficient. In the kitchen, there's the concept of *mise en place*: have all the ingredients at hand, and ready, so that the cooking is efficient and requires a lot less running around the kitchen. So we get all the ingredients up front, then cook. If the ingredients are data elements and cooking is rendering, it becomes clear how to map this to page loads. Having just one trip to fetch the data is much better. We do that, of course, by reading the whole recipe (page) first, to have an up-front list of exactly what we need.

It is important to note that it's the sequential nature of requests here that is the problem, not the total number of requests. If there were three concurrent fetches (send kitchen employees to get onions, oil, and salt in parallel) then there would be no waterfall. But fetching each thing sequentially is where the problem lies.

Rather like the $N + 1$ problem discussed much earlier, it is unlikely anyone sets out with the intention to write a waterfall. Instead, it tends to happen when there are nested components that depend on their parent element. So if the parent element is the user account, maybe the child elements show data based on what features the user account uses. What to do depends on whether we need the parent element's data, or whether we need the parent element to be rendered.

If the parent element's data is needed, push this into the backend. Let the server figure out how to deliver all of the data in one shot either itself or by pushing the request down to the database (ideally, avoiding the $N + 1$ issue in either place). This does increase the time it takes to get data back, which can delay the first arrival of data, but if used wisely on the LCP content, it reduces the total time.

If it's the parent rendering that matters, then the strategy is *mise en place*: read the whole data tree (recipe) first to find out exactly what to ask for; then start the rendering. Waiting for the parent rendering is not a true dependency, because the content of the parent is not actually needed to render the child elements – but the request to get the data for the child elements waits for the parent. Don't wait.

Rendering Pipeline

Once the data is loaded, the next part we need to be concerned about is how to actually render this efficiently. If we assume that the user's display refreshes 60 times per second, a frame render needs to take place in about 10 ms, otherwise the page stutters (and this is sometimes referred to as "jank") [Lew23].

An important limiting factor is the browser itself: it uses one thread for parsing, JavaScript/CSS evaluation, and rendering [WBKW13]. So, parallelism does not help much here, though compositing can be done in a different thread and sends the results to the GPU. Producing a frame goes through five discrete steps: JavaScript, Style, Layout, Paint, Composite. Let's look at each of those [Lew23]:

1. JavaScript is code, obviously, that results in changes to the visual layout.
2. Style calculations figure out what CSS rules apply to what elements.
3. Layout is figuring out the size of elements and placing them on the screen.
4. Paint is the process of filling in the pixels: text, colours, images, etc.

5. Composite is the step of putting all the layers of the page together, making sure that overlapping elements go in the correct front-to-back order.

Whenever there is a change, the browser starts at the latest stage possible to execute that change. So if the colour of some element changes, the next frame starts from the Style step of the process and skips layout. But if a button's width changes, then the Layout step is done as well as Paint and Composite. If we want to reduce the rendering time to avoid jank, the goal is to make it so changes affect the fewest steps possible.

Consider a concrete example about a text shimmer animation[1]. If the code changes background-position, this forces a repaint on every frame, so it requires more CPU time. If instead the approach is a CSS mask that animates a transform, the text is painted one time and the work is just composition: slide the overlay over a bit and stack the elements again. That's not for free, though, because the overlay needs to have its own layer, which takes up more memory.

Something that sounds the same, but is actually different, is *cumulative layout shift*. Perhaps you've had the negative experience on a webpage where you go to click on something and the page moves or shifts and you end up clicking the wrong thing? You probably hate that. Sometimes things moving around cannot be avoided, but it's still unpleasant. Your code can and should change the layout of the page, sometimes, to adapt to the information shown (e.g., a big image arriving late), but ideally space is reserved for it so that doesn't happen. In this section, we're focusing on preventing unnecessary re-rendering work rather than stopping things from moving on the page.

If big changes are not necessary, don't do them. We can think of the previous frame not only as the initial state, but also as cached results. If there are no changes, we can just use that. If there are minor changes (e.g., composition), we invalidate the parts of the cache that have changed but no more than that. If there are major changes (e.g., layout changes), we invalidate the whole cache and it forces a more expensive operation.

Layout thrashing is the term for many forced layouts in quick succession. The most efficient way for the browser to do the work it needs is to wait for all the changes to take place, then do the layout once. If we don't do that, we end up repeating a lot of work unnecessarily. Consider this (inefficient) code and its improved version below [WLP15].

```
function resizeAllParagraphsToMatchBlockWidth () {
  // Puts the browser into a read-write-read-write cycle.
  for (let i = 0; i < paragraphs.length; i++) {
    paragraphs[i].style.width = `${box.offsetWidth}px`;
  }
}
```

This is inefficient because every iteration of the loop assigns the width of a paragraph, which forces a new layout on the next iteration of the loop, because the read of the `box.offsetWidth` value can only be accurate after previous layout changes have taken place. The size of the paragraph may affect the size of the box, and the browser does not know that for sure until it re-runs the layout process. You as the developer might know that the paragraph size should not change the box width, so you can fix that by reading the value once, outside the loop:

```
// Read.
const width = box.offsetWidth;

function resizeAllParagraphsToMatchBlockWidth () {
  for (let i = 0; i < paragraphs.length; i++) {
    // Now write.
    paragraphs[i].style.width = `${width}px`;
  }
}
```

This version looks a lot better. We read the value once, so at worst there is one forced layout there. Then that value is used in making all the changes before we go on. If I send a document to a colleague to review, it is, after all, more efficient if they give me all the changes I should make and I make them in one round, rather than back-and-forth on every individual comment in the document.

[1] `https://x.com/devongovett/status/2092991811157463500`

All of this describes the work required to render one frame! A modern page that has a lot of data may have hundreds or thousands of elements and the data behind them is changing. We need a strategy for managing that.

Modelling

A webpage has a *Document Object Model (DOM)* that represents the structure of the page. A more complex page has a more complex model, with more elements in it. We will consider three ideas around improving the page speed: simplifying, reducing the rendering cost, and reducing the cost of changes. Google's Lighthouse performance tool considers the DOM to be a warning-level of complex at 800 nodes and an error-level of complex at 1400.

It seems only sensible that it then takes more work to draw the page if it's more complex. Thus, simplifying the DOM is an obvious strategy: if there's less loaded on the page, there's less to do. This may also result in less data being required from the backend, which then improves the data loading time. Two for one. But if you need all the data on the page, it's hard to cut this down.

If we can't reduce the data, but want to speed up rendering, the plan is *virtualization*, a term that is, sadly, overloaded with a bunch of other meanings in computing. In this context, it means only creating elements for things that are visible on screen. If the page is a calendar for the month and there are hundreds of appointments but only five fit on any given day, only make elements for five per day, and if there are more, let the user scroll and the elements get added as they scroll. Otherwise, lots of effort is wasted on boxes that do not appear on the screen. The tradeoff, though, is that elements that are virtualized are not in the DOM so we can't find them with a search.

If we want to render those elements, we could split that off so it happens after the LCP so the user thinks all is rendered that much sooner. There exists the `content-visibility` property in CSS that lets things be rendered on demand; it reduces the work done on an initial page render and skips re-rendering for things that aren't visible [Wag23].

If one data element is updated, there is a risk that it forces a very large re-load and re-render. If the element is part of a collection of a thousand entries, it would be wasteful to re-draw all entries if only one of them has changed. If this is a calendar, a change in one appointment from 9:00 to 9:30 should result in only updating that appointment (and the day it's on) rather than all appointments. If subscribed to the entire collection, the frontend sees that the whole collection has changed and must either re-draw everything, or go through the entire collection to see which ones have changed. React's docs say that the general tree comparison has $O(n^3)$ runtime behaviour so it does something that's "only" $O(n)$, but that's still expensive for a large collection.

Subscribing to the entire collection results in something that resembles false-sharing: something is re-rendered because something near it has changed, rather than the thing itself.

If instead, appointments are in a lookup table by ID and there's a separate list of IDs, the calendar component needs to look at the list of IDs and create one component per ID; that component gets its data from the table. So moving one appointment changes one entry in the table and everything else remains unchanged. It does add complexity of subscribing to more things and getting multiple change notifications, but if relatively few items change at a time, this is worth it.

What makes for good IDs? If the data element itself has a unique ID (e.g., account number), that is a good choice because it is straightforward to know which account to update when a change notification is received. If we go by position in the list, that might be frustrating because it makes re-ordering a pain. Random or unstable IDs are the worst choice because we could end up with a lot of unnecessary re-creation of elements [Met23].

Frontend is Not So Different

While the idea of trying to get the LCP done earlier and then call that "done" is a little bit unusual, everything else about frontend performance relates nicely to other things we've covered in the course: move things off the critical

path, do less work, and trade space for time.

References

[D20] Deloitte Digital and 55. Milliseconds make millions: A study on how improvements in mobile site speed positively affect a brand's bottom line. Technical report, Deloitte Ireland LLP, 2020. Commissioned by Google. URL: https://www.thinkwithgoogle.com/_qs/documents/9757/Milliseconds_Make_Millions_report_hQYAbZJ.pdf.

[KS12] S. Shunmuga Krishnan and Ramesh K. Sitaraman. Video stream quality impacts viewer behavior: Inferring causality using quasi-experimental designs. In *Proceedings of the 2012 Internet Measurement Conference (IMC '12)*, 2012. doi:10.1145/2398776.2398799.

[KSNH15] Hammad Khalid, Emad Shihab, Meiyappan Nagappan, and Ahmed E. Hassan. What do mobile app users complain about? *IEEE Software*, 32(3):70–77, 2015. doi:10.1109/MS.2014.50.

[Lew23] Paul Lewis. Rendering performance, 2023. Online; accessed 2026-11-11. URL: https://web.dev/articles/rendering-performance.

[Met23] Meta Platforms, Inc. Reconciliation, 2023. Online; accessed 2026-11-11. URL: https://legacy.reactjs.org/docs/reconciliation.html.

[Wag22] Jeremy Wagner. Don't fight the browser preload scanner, 2022. Online; accessed 2026-11-11. URL: https://web.dev/articles/preload-scanner.

[Wag23] Jeremy Wagner. How large DOM sizes affect interactivity, and what you can do about it, 2023. Last updated May 9, 2023. Online; accessed 2026-11-11. URL: https://web.dev/articles/dom-size-and-interactivity.

[Wal23] Philip Walton. First contentful paint (fcp), 2023. Last updated December 6, 2023. Online; accessed 2026-11-11. URL: https://web.dev/articles/fcp.

[WBKW13] Xiao Sophia Wang, Aruna Balasubramanian, Arvind Krishnamurthy, and David Wetherall. Demystifying page load performance with WProf. In *10th USENIX Symposium on Networked Systems Design and Implementation (NSDI '13)*, pages 473–485. USENIX Association, 2013. URL: https://www.usenix.org/system/files/conference/nsdi13/nsdi13-final177.pdf.

[WLP15] Jeremy Wagner, Paul Lewis, and Barry Pollard. Avoid large, complex layouts and layout thrashing, 2015. Published March 20, 2015; last updated May 7, 2025. Online; accessed 2026-11-11. URL: https://web.dev/articles/avoid-large-complex-layouts-and-layout-thrashing.

[WP25] Philip Walton and Barry Pollard. Optimize largest contentful paint, 2025. Online; accessed 2026-11-11. URL: https://web.dev/articles/optimize-lcp.